

Skills MCP - Protocol Parity Implementation Plan

Concrete code changes to align the Skills MCP with Databricks Genie Code’s published protocol

Author: Cursor (Composer 2.5) Date: 2026-06-03 Companion to: `skills-mcp-routing-failure-analysis-2026-06-03.pdf`

TL;DR

This document specifies the code changes that close the protocol gap between our Skills MCP and Databricks Genie Code. Three phases, each shippable independently, each behind a feature flag matching the existing `SKILLS_SUB_PASS_ENABLED` rollout discipline.

Phase	Scope	Risk	Ship when
1	Low-risk fixes that compound (sub_k default, banner, IDENTITY-003 dedupe, tool-description update)	Low	Immediately
2	Routing-protocol delivery + progressive disclosure (the centerpiece)	Medium - new code paths, feature-flagged	After unit tests + staging validation
3	Default progressive disclosure to ON, retire bulk-dump path	Low - flag flip + dead-code removal	After 2-week staging burn-in

The implementation reuses your existing patterns: dataclass-frozen policy objects, env-var settings with range validation in `Settings.from_env`, feature-flag-default-False, and dual-format response payloads (off-path returns the legacy shape byte-for-byte).

Repositories touched

- `eToro/DataPlatform/databricks/skills-mcp/databricks-skills-mcp` (the Skills MCP server)
- `eToro/DataPlatform/databricks/skills-mcp/databricks-mcp-gateway` (the gateway middleware)
- `eToro/DataPlatform/databricks/data-skills/scripts` (the validator)
- `eToro/DataPlatform/databricks/data-rules` (already exists - source of `.assistant_workspace_instructions.md`)

Phase 1 - Low-risk compounding fixes

These four changes are independent of progressive disclosure and can ship as one PR.

1.1 Bump `SKILLS_SUB_K_DEFAULT` from 1 to 3

File: `databricks/skills-mcp/databricks-skills-mcp/app.yaml`

```
# v1 hub-and-spoke second pass (DEI-3745). Production default raised
# from 1 to 3 (DA-XXXX) so the LLM sees the top three matched
# children per hub instead of a top-1 coin flip. Token cost is small
# (3x ~1500-char child bodies per hub); recall benefit on
# definition-sensitive questions ("how many funded?") is large.
- name: SKILLS_SUB_K_DEFAULT
  value: "3"
```

The validation in `Settings.from_env` already clamps to `[0, 3]` so no `settings.py` change is needed.

1.2 Add a `find_skills` banner to the gateway middleware

File: `databricks/skills-mcp/databricks-mcp-gateway/server/middleware.py`

Add a third banner constant alongside `_BANNER_SQL` and `_BANNER_GENIE`:

```
_BANNER_FIND_SKILLS = (
    "PROGRESSIVE DISCLOSURE - this tool returns DISCOVERY-level results "
    "(skill name + description + match score). To activate a skill and "
    "read its full body for grounding, call `skills_get_skill(id)` on "
    "the matched skill BEFORE writing SQL. If the response includes a "
    "`routing_protocol` field, treat it as authoritative routing rules "
    "(equivalent to the Databricks Assistant's workspace instructions)."
)
```

Extend `_GUARDED_TOOLS` to include `skills_find_skills`:

```
_GUARDED_TOOLS = frozenset(
    {
        "databricks_ops_ask_genie",
        "databricks_sql_execute_sql",
        "databricks_sql_execute_sql_read_only",
        "databricks_ops_execute_sql",
        "databricks_ops_execute_sql_multi",
        "skills_find_skills",
    }
)
```

Update the banner-selection logic in `ToolDescriptionRewriterMiddleware.on_list_tools`:

```

if tool.name == "databricks_ops_ask_genie":
    banner = _BANNER_GENIE
elif tool.name == "skills_find_skills":
    banner = _BANNER_FIND_SKILLS
else:
    banner = _BANNER_SQL

```

This banner survives HTTP transport (Anthropic clients drop `instructions` but render tool descriptions normally - documented in `instructions.py` line 4).

1.3 Strengthen the `find_skills` tool description

File: `databricks/skills-mcp/databricks-skills-mcp/server/tools.py`

Replace the existing docstring (lines 152-217) opening paragraphs:

```

@mcp_server.tool
def find_skills(
    question: str,
    k: int = 5,
    domain_tag: str | None = None,
    sub_k: int | None = None,
) → dict:
    """Find the top-K skills for a natural-language eToro business question.

    USE FIRST when the user asks a business question about eToro data
    (metrics, KPIs, trends, accounts, deposits, fees, GLA, GMV, regions,
    instruments).

    TWO-STEP GROUNDING PROTOCOL (matches Databricks Genie Code / Anthropic
    Agent Skills):
    1. Call `find_skills(question)` to discover candidate skills.
       The response includes name, description, score, and a
       `routing_protocol` field carrying the workspace's
       authoritative routing rules.
    2. If the question matches one of the candidate skills' descriptions
       (or its `triggers`), call `get_skill(id)` on that skill to
       ACTIVATE it - load its full body for grounding. THEN run via
       `databricks_sql_execute_sql`.

    DO NOT write SQL directly from `find_skills` results - the description
    is for routing only. Activation via `get_skill` is required for the
    full grounding artifact.

    SKIP for raw SQL execution, schema browsing (SHOW/DESCRIBE/EXPLAIN),
    Unity Catalog navigation, or SQL syntax help - call `databricks_sql_*`
    directly in those cases.

    Returns *advisory* metadata only - never executes SQL, never returns rows.
    """

```

Keep the rest of the docstring (Args, Returns) as-is.

1.4 Extend IDENTITY-003 to dedupe sub-skill stems

File: `databricks/data-skills/scripts/validate_skills.py`

Today the dedupe only collects entry-point stems (line 844-846). The MCP loader at `loader.py` lines 474-488 dedupes the union of hub stems + sub-skill stems - the validator must mirror that. Around line 864:

```
# Cross-corpus uniqueness on the union of hub stems AND sub-skill
# stems. The MCP's loader.py dedupes both namespaces under one key
# (hub stems and sub-skill stems share the global slug keyspace
# because get_skill resolves either by slug). The validator must
# mirror that contract or skills-CI passes while the MCP fails to
# boot - exactly the gap that caused the dashboard-queries /
# data-patterns / metric-definitions collision incident on
# spaceship + moneyfarm + options.
sub_stems = [_sub_skill_stem(p) for p in sub_skill_files]
union_counts = Counter(stems + sub_stems)
duplicates = sorted(s for s, c in union_counts.items() if c > 1)
for dup in duplicates:
    msg = f"duplicate stem across corpus (hubs U sub-skills): {dup!r}"
    all_errors.append(msg)
    findings.append(
        ValidationFinding(
            check_id="IDENTITY-003",
            check_name="unique_stem",
            severity="error",
            skill=dup,
            message=msg,
            file="",
        )
    )
```

Helper:

```
def _sub_skill_stem(path: Path) → str:
    """Sub-skill identity is the file stem (e.g. 'mimo-panel-and-ddr.md'
    → 'mimo-panel-and-ddr'). Mirrors loader.py's identity rule."""
    return path.stem
```

Existing tests in `tests/test_validate_skills.py` need a new case for hub-vs-child collision; the failure mode is currently invisible to CI but caught by the MCP loader at boot.

Phase 2 - Protocol parity (routing protocol + progressive

disclosure)

This is the centerpiece. Two feature flags so each piece can be rolled out independently:

- `SKILLS_ROUTING_PROTOCOL_ENABLED` (default False) - when True, `find_skills` includes a `routing_protocol` field in every response, populated from `databricks/data-rules/.assistant_workspace_instructions.md` in the same monorepo. Off-path returns the legacy shape byte-for-byte.
- `SKILLS_PROGRESSIVE_DISCLOSURE_ENABLED` (default False) - when True, `find_skills` returns DISCOVERY-only results (name + description + score, no body_markdown, no nested sub-skill bodies). `get_skill(id)` becomes the activation channel that returns the full body. Off-path returns the existing v1 sub-pass shape.

Both flags follow the same operational discipline as `SKILLS_SUB_PASS_ENABLED`: code lands dark, ops flips the env var when ready, no schema migration required.

2.1 New module - the routing protocol loader

File: `databricks/skills-mcp/databricks-skills-mcp/server/routing_protocol.py` (new)

```

"""Load the workspace routing protocol from the data-rules subfolder.

Mirrors how Databricks Genie Code consumes
`/Workspace/.assistant_workspace_instructions.md` - a workspace-scoped,
imperative routing-rules document loaded into every interaction's
context.

Our equivalent is authored at
`databricks/data-rules/.assistant_workspace_instructions.md` in the
DataPlatform monorepo and shipped to `/Workspace/...` by skills-cd. The
Skills MCP clones the same monorepo (see settings.SKILLS_REPO_URL +
SKILLS_REPO_SUBDIR), so we can read the same file from disk after
git_sync completes - no second fetch, no auth, no drift.

Rendered into the `find_skills` response as a top-level
`routing_protocol` string so external MCP clients (Cursor, Claude Code,
Claude Desktop) see what the Databricks Assistant already sees.
Anthropic's MCP `instructions` field is silently dropped on HTTP
transport (see databricks-mcp-gateway/server/instructions.py line 4) -
this is the data-channel workaround.
"""
from __future__ import annotations

import logging
from dataclasses import dataclass
from pathlib import Path

logger = logging.getLogger(__name__)

# Same 4000-char cap Databricks enforces on workspace instructions
# (per the Genie Code blog). Keeps the response payload bounded and
# matches the constraint authors already write to.
_MAX_PROTOCOL_CHARS = 4000

@dataclass(frozen=True)
class RoutingProtocol:
    """The loaded routing protocol document."""
    content: str
    source_path: str
    char_count: int

    @classmethod
    def empty(cls) → RoutingProtocol:
        return cls(content="", source_path="", char_count=0)

    @property
    def is_empty(self) → bool:
        return self.char_count == 0

def load_routing_protocol(
    repo_root: Path,

```

```

*,
relative_path: str = "databricks/data-rules/.assistant_workspace_instructions.md",
) → RoutingProtocol:
    """Read the routing protocol from the cloned monorepo.

    Returns ``RoutingProtocol.empty()`` if the file is missing – the MCP
    is still useful without it; the LLM just won't get the workspace
    routing rules. Logs a warning so operators see the absence.

    Truncates to ``_MAX_PROTOCOL_CHARS`` if the file grows past the cap;
    truncation logs a warning so authors get a signal to trim.
    """
    full_path = repo_root / relative_path
    if not full_path.is_file():
        logger.warning(
            "routing protocol file not found at %s; "
            "find_skills responses will not include routing rules",
            full_path,
        )
        return RoutingProtocol.empty()

    text = full_path.read_text(encoding="utf-8")
    original_len = len(text)
    if original_len > _MAX_PROTOCOL_CHARS:
        text = text[:_MAX_PROTOCOL_CHARS]
        logger.warning(
            "routing protocol at %s is %d chars; truncated to %d. "
            "Consider trimming the source file.",
            full_path, original_len, _MAX_PROTOCOL_CHARS,
        )

    return RoutingProtocol(
        content=text,
        source_path=str(relative_path),
        char_count=len(text),
    )

```

2.2 Settings extensions

File: `databricks/skills-mcp/databricks-skills-mcp/server/settings.py`

Add to the `Settings` dataclass after the `skills_sub_*` block:

```

# Routing protocol delivery (Phase 2.1 of MCP/Genie parity).
# Loads `databricks/data-rules/.assistant_workspace_instructions.md`
# from the cloned monorepo and surfaces it on every find_skills
# response. Off-by-default; flip the env var to land dark first.
skills_routing_protocol_enabled: bool = False
skills_routing_protocol_path: str = (
    "databricks/data-rules/.assistant_workspace_instructions.md"
)

# Progressive disclosure (Phase 2.2 of MCP/Genie parity). When
# True, find_skills returns DISCOVERY-only results (name +
# description + score) and get_skill is the ACTIVATION channel
# for full bodies. Matches Anthropic's Agent Skills 3-stage
# spec and the Databricks Genie Code skill loading model. Off-by-
# default; off-path returns the legacy v1 sub-pass payload byte-
# for-byte (existing audit baselines unchanged).
skills_progressive_disclosure_enabled: bool = False

```

In `Settings.from_env`, after the `skills_sub_min_score` parsing:

```

skills_routing_protocol_enabled = _env_bool(
    "SKILLS_ROUTING_PROTOCOL_ENABLED", False,
)
skills_routing_protocol_path = _env(
    "SKILLS_ROUTING_PROTOCOL_PATH",
    "databricks/data-rules/.assistant_workspace_instructions.md",
) or "databricks/data-rules/.assistant_workspace_instructions.md"

skills_progressive_disclosure_enabled = _env_bool(
    "SKILLS_PROGRESSIVE_DISCLOSURE_ENABLED", False,
)

```

And include them in the `cls(...)` constructor call at the bottom.

2.3 Wire the loader through `app.py`

File: `databricks/skills-mcp/databricks-skills-mcp/server/app.py`

After the `_skill_index = SkillIndex()` line:

```

from .routing_protocol import RoutingProtocol, load_routing_protocol

# Loaded at startup, refreshed on every rebuild_index() cycle so that
# `data-rules/.assistant_workspace_instructions.md` edits go live on
# the same poll interval as skill edits.
_routing_protocol: RoutingProtocol = RoutingProtocol.empty()

```

In `rebuild_index()`, after the `corpus_root` calculation (line 66):


```

# Load the routing protocol from the SAME repo root (not the
# corpus subdir). The protocol lives at
# `databricks/data-rules/.assistant_workspace_instructions.md`
# at the monorepo root; the corpus is at
# `databricks/data-skills/skills/`. Both ride the same git_sync.
global _routing_protocol
if _settings.skills_routing_protocol_enabled:
    _routing_protocol = await loop.run_in_executor(
        None,
        load_routing_protocol,
        repo_dir,
        _settings.skills_routing_protocol_path,
    )
else:
    _routing_protocol = RoutingProtocol.empty()

```

In the `register_tools(...)` call:

```

register_tools(
    mcp_server,
    skill_index=_skill_index,
    embedder=_embedder,
    routing_protocol_provider=lambda: _routing_protocol, # NEW
    cutoff_policy=CutoffPolicy(...),
    sub_pass_policy=SubPassPolicy(...),
    progressive_disclosure_policy=ProgressiveDisclosurePolicy( # NEW
        enabled=_settings.skills_progressive_disclosure_enabled,
    ),
)

```

The `routing_protocol_provider` is a callable rather than a value so the tool sees the latest after every `rebuild_index()` swap without a re-registration.

2.4 New `ProgressiveDisclosurePolicy` + injection

File: `databricks/skills-mcp/databricks-skills-mcp/server/tools.py`

Add a third dataclass alongside `CutoffPolicy` and `SubPassPolicy`:

```
@dataclass(frozen=True)
class ProgressiveDisclosurePolicy:
    """Knobs for the Phase-2 progressive-disclosure response shape.

    When ``enabled = False`` (default), find_skills returns the existing
    v1 sub-pass payload byte-for-byte (or the pre-v1 hub-only payload
    when SubPassPolicy.enabled is also False). The off-path is a
    rollback safety net.

    When ``enabled = True``, find_skills returns DISCOVERY-stage results
    only:
        - skills: list of {id, name, description, score, hint, kind,
                          parent_skill_id?, triggers}
        - routing_protocol: the workspace routing rules document
        - usage / metadata fields preserved

    No body_markdown, no nested sub_skills, no example_sql in the
    discovery response. ACTIVATION is the caller's responsibility:
    after picking a skill from discovery, call get_skill(id) for the
    full body.

    The Skill model itself is unchanged; this is purely a response-
    shape policy. The index, the embedder, the loader all stay as-is.
    """
    enabled: bool = False
```

Update `register_tools` signature:

```
def register_tools(
    mcp_server,
    *,
    skill_index: SkillIndex,
    embedder: Embedder,
    routing_protocol_provider=lambda: None, # NEW
    acl_filter=filter_by_acl,
    acl_filter_with_subs=filter_by_acl_with_subs,
    cutoff_policy: CutoffPolicy | None = None,
    sub_pass_policy: SubPassPolicy | None = None,
    progressive_disclosure_policy: ProgressiveDisclosurePolicy | None = None, # NEW
) -> None:
    ...
    pd_policy = progressive_disclosure_policy or ProgressiveDisclosurePolicy()
```

2.5 `find_skills` - new discovery shape (when enabled)

File: `databricks/skills-mcp/databricks-skills-mcp/server/tools.py`

Inside `find_skills`, after the existing `wide_k`/`hits` computation but BEFORE the off-path branch into `_build_hub_only_response`:

```

# Phase 2: progressive-disclosure on-path. Returns DISCOVERY-
# stage skill summaries (name + description + score) for top-K,
# treating hubs and matched sub-skills as first-class candidates.
# No body_markdown is included. The LLM activates via get_skill.
if pd_policy.enabled:
    return _build_progressive_disclosure_response(
        ranked_skills=ranked_skills,
        hub_scores=hub_scores,
        skill_index=skill_index,
        query_vec=query_vec,
        acl_filter=acl_filter,
        acl_filter_with_subs=acl_filter_with_subs,
        k=k,
        sub_k=sub_k if sub_k is not None else sub_policy.k_default,
        policy=policy,
        sub_min_score=sub_policy.min_score,
        routing_protocol=routing_protocol_provider(),
        embedder=embedder,
    )

```

The new helper at module scope:

```

def _build_progressive_disclosure_response(
    *,
    ranked_skills: list[Skill],
    hub_scores: dict[str, float],
    skill_index: SkillIndex,
    query_vec,
    acl_filter,
    acl_filter_with_subs,
    k: int,
    sub_k: int,
    policy: CutoffPolicy,
    sub_min_score: float,
    routing_protocol,
    embedder,
) → dict:
    """Discovery-stage response: name + description + score for top-K
    skills, where 'skill' means hub OR matched sub-skill. No bodies.

    The LLM uses this to pick which skill to ACTIVATE via get_skill.
    Mirrors the Anthropic Agent Skills 3-stage spec (discovery,
    activation, execution) that Databricks Genie Code implements.

    Children (sub-skills) appear as first-class entries with
    ``kind = "sub_skill"`` and ``parent_skill_id`` set to the owning
    hub. Hubs appear with ``kind = "hub"``. Both are sorted by score
    in one flat list - no nesting.
    """
    # Score each hub's children against the query - reuses the existing
    # search_subs_under helper (adds zero new index machinery).
    candidates: list[tuple[str, str, float, str | None, Skill | object]] = []
    # tuple: (kind, id, score, parent_id_or_None, payload_for_serialization)
    for hub in ranked_skills:
        candidates.append(("hub", hub.id, hub_scores[hub.id], None, hub))
        if sub_k > 0:
            for hit in skill_index.search_subs_under(hub.id, query_vec, sub_k):
                candidates.append(
                    ("sub_skill", hit.sub_skill.id, hit.score, hub.id, hit.sub_skill)
                )

    # ACL filter: each sub-skill inherits its hub's ACL via the union
    # filter. Hubs themselves are filtered as today.
    visible_hubs = {s.id for s in acl_filter(ranked_skills)}
    candidates = [
        c for c in candidates
        if (c[0] == "hub" and c[1] in visible_hubs)
        or (c[0] == "sub_skill" and c[3] in visible_hubs)
    ]

    # Sort flat by score, apply gap cut + floor split
    candidates.sort(key=lambda t: t[2], reverse=True)

    floor = policy.min_score
    top_score_raw = candidates[0][2] if candidates else None

```

```

# Reuse existing _largest_gap_cut and floor logic by projecting
# to (Skill, score) - we'll re-attach kind/parent after.
projected = [(c[4], c[2]) for c in candidates]
projected = _largest_gap_cut(
    projected,
    search_window=policy.search_window,
    buffer=policy.buffer,
)
if floor > 0 and projected:
    above = [p for p in projected if p[1] ≥ floor]
    if above:
        projected = above
projected = projected[:k]

# Re-pair with kind/parent metadata (id-keyed lookup on the
# original candidates list)
by_id = {c[1]: c for c in candidates}
skills_payload = []
for skill_obj, score in projected:
    kind, sid, _, parent_id, _ = by_id[skill_obj.id]
    # Discovery shape: NAME + DESCRIPTION + SCORE ONLY. No body.
    if kind == "hub":
        skill_dict = {
            "id": skill_obj.id,
            "name": getattr(skill_obj, "name", None) or skill_obj.id,
            "description": skill_obj.description,
            "kind": "hub",
            "domain_tags": skill_obj.domain_tags,
            "triggers": skill_obj.triggers,
            "unity_catalog_assets": skill_obj.unity_catalog_assets,
            "score": score,
            "match_quality_hint": _per_skill_hint(score, floor),
        }
    else:
        skill_dict = {
            "id": skill_obj.id,
            "description": skill_obj.description,
            "kind": "sub_skill",
            "parent_skill_id": parent_id,
            "triggers": skill_obj.triggers,
            "unity_catalog_assets": skill_obj.unity_catalog_assets,
            "score": score,
            "match_quality_hint": _per_skill_hint(score, sub_min_score),
        }
    skills_payload.append(skill_dict)

call_hint = _per_call_hint([(s, sc) for s, sc in projected], floor)
all_below = bool(
    floor > 0 and projected and all(sc < floor for _, sc in projected)
)

response = {

```

```

    "skills": skills_payload,
    "filtered_count": len(ranked_skills) - len(visible_hubs),
    "top_score": top_score_raw,
    "match_quality_hint": call_hint,
    "all_below_floor": all_below,
    "advisory": _ADVISORY_BY_HINT[call_hint],
    "protocol": "progressive_disclosure_v1", # explicit shape tag
    "next_step": (
        "To activate a skill, call get_skill(id) on the highest-"
        "scoring entry whose description matches the user's "
        "question. Do NOT generate SQL from this discovery response."
    ),
    "usage": {"total_tokens": embedder.last_usage_total_tokens},
}

if routing_protocol is not None and not routing_protocol.is_empty:
    response["routing_protocol"] = {
        "content": routing_protocol.content,
        "source": routing_protocol.source_path,
        "instruction": (
            "These are the workspace's authoritative routing rules. "
            "Apply them BEFORE picking a skill - they map question "
            "patterns to specific skills and tables, and forbid "
            "common conflations (e.g. 'Funded' vs 'FTD'). Equivalent "
            "to the Databricks Assistant's workspace instructions."
        ),
    }

return response

```

2.6 `find_skills` - routing_protocol on the legacy paths too

Even with `SKILLS_PROGRESSIVE_DISCLOSURE_ENABLED=False`, ship the routing protocol when `SKILLS_ROUTING_PROTOCOL_ENABLED=True`. Surgical injection in `_build_hub_only_response` and the existing on-path response builder:

```

def _attach_routing_protocol(response: dict, routing_protocol) → dict:
    """Mutate-and-return helper. Adds routing_protocol to any response
    shape (hub-only, sub-pass, progressive). Idempotent on empty
    protocol (key omitted)."""
    if routing_protocol is not None and not routing_protocol.is_empty:
        response["routing_protocol"] = {
            "content": routing_protocol.content,
            "source": routing_protocol.source_path,
        }
    return response

```

Then `_build_hub_only_response` returns `_attach_routing_protocol(response_dict, routing_protocol)`, and the on-path return at the bottom of `find_skills` does the same. This way, Phase-1 deployments can ship the routing protocol without buying into progressive disclosure yet.

2.7 `get_skill` is already activation-shaped

The existing `get_skill(id)` (tools.py lines 457-493) already returns the full skill body via `_skill_to_dict`, AND already resolves sub-skill stems globally with `parent_skill_id` and `kind: "sub_skill"`. **No change needed for activation semantics.**

Add one paragraph to its docstring so the contract is explicit:

```
@mcp_server.tool
def get_skill(id: str) → dict:
    """Activate one skill (or sub-skill) by id, returning its full body.

    ACTIVATION stage of the Anthropic Agent Skills 3-stage protocol
    (discovery via find_skills, activation via get_skill, execution
    via the returned grounding artifacts). Call this AFTER find_skills
    when you have identified which skill matches the user's question.

    Args:
        id: The skill or sub-skill slug ...
```

The rest of the docstring stays.

Phase 3 - Cleanup (after Phase 2 stabilizes)

3.1 Default progressive disclosure to True

Once Phase 2 has run for ~2 weeks in stg with healthy audit metrics (no 500s, no client-side fall-through to bulk SQL, downstream LLM behavior correct), flip the default in `app.yaml`:

```
- name: SKILLS_PROGRESSIVE_DISCLOSURE_ENABLED
  value: "true"
- name: SKILLS_ROUTING_PROTOCOL_ENABLED
  value: "true"
```

The `Settings` defaults stay False so local dev and tests behave like the legacy path until they explicitly enable the new shape. Operators control rollout via env.

3.2 Retire bulk-dump path (optional, after Phase 3.1 + 30 days)

After progressive disclosure has been the default for 30 days with no rollback events, the legacy path code can be removed:

- Delete `_build_hub_only_response` from `tools.py`
- Delete the on-path sub-skill bulk-dump branch (`tools.py` lines 287-435 except the routing-protocol attachment)

- Drop `SubPassPolicy` (or merge its knobs into `ProgressiveDisclosurePolicy`)

This is a code-deletion-only PR with no behavior change for clients that have already migrated. Defer until the burn-in window is clean.

3.3 data-rules linter additions (low priority)

`databricks/data-rules/scripts/lint_rules.py` already enforces imperative form on the routing-protocol document. Add one new check:

```
# Cross-reference check: every skill_id mentioned in the routing
# protocol must exist in the skills corpus. Catches stale references
# after a skill is renamed (e.g. customer-populations was renamed
# to customer-populations-and-lifecycle on DA-72; the routing
# protocol must follow). Run only when both repos are checked out.
```

This is defensive and not blocking on the protocol-parity rollout.

Tests

4.1 Unit tests for `routing_protocol.py`

File: `databricks/skills-mcp/databricks-skills-mcp/tests/test_routing_protocol.py` (new)


```

"""Tests for the routing protocol loader (Phase 2.1)."""
from pathlib import Path

import pytest

from server.routing_protocol import (
    RoutingProtocol,
    load_routing_protocol,
    _MAX_PROTOCOL_CHARS,
)

def test_load_routing_protocol_happy_path(tmp_path):
    rules_dir = tmp_path / "databricks" / "data-rules"
    rules_dir.mkdir(parents=True)
    target = rules_dir / ".assistant_workspace_instructions.md"
    target.write_text("# Rules\n- Treat Funded as IsFunded=1\n", encoding="utf-8")

    rp = load_routing_protocol(tmp_path)

    assert rp.is_empty is False
    assert "Funded" in rp.content
    assert rp.source_path.endswith(".assistant_workspace_instructions.md")

def test_load_routing_protocol_missing_file_returns_empty(tmp_path, caplog):
    rp = load_routing_protocol(tmp_path)
    assert rp.is_empty is True
    assert any("not found" in r.message for r in caplog.records)

def test_load_routing_protocol_truncates_oversize(tmp_path, caplog):
    rules_dir = tmp_path / "databricks" / "data-rules"
    rules_dir.mkdir(parents=True)
    target = rules_dir / ".assistant_workspace_instructions.md"
    target.write_text("x" * (_MAX_PROTOCOL_CHARS + 100), encoding="utf-8")

    rp = load_routing_protocol(tmp_path)

    assert rp.char_count == _MAX_PROTOCOL_CHARS
    assert any("truncated" in r.message for r in caplog.records)

def test_routing_protocol_empty_factory():
    rp = RoutingProtocol.empty()
    assert rp.is_empty is True
    assert rp.content == ""
    assert rp.char_count == 0

```

4.2 Tool tests - the new shapes

File: `databricks/skills-mcp/databricks-skills-mcp/tests/test_tools.py` (extend)

Three new tests covering the response-shape feature flags:

```
def test_find_skills_routing_protocol_attached_when_enabled(tools_setup):
    """When SKILLS_ROUTING_PROTOCOL_ENABLED is True, every find_skills
    response carries routing_protocol regardless of other flags."""
    response = tools_setup.find_skills(
        "how many funded customers?",
        # routing_protocol_provider returns a non-empty fake
        # via the fixture
    )
    assert "routing_protocol" in response
    assert "Funded" in response["routing_protocol"]["content"]

def test_find_skills_routing_protocol_omitted_when_disabled(tools_setup_disabled):
    """Off-path: routing_protocol field is absent from response."""
    response = tools_setup_disabled.find_skills("how many funded customers?")
    assert "routing_protocol" not in response

def test_find_skills_progressive_disclosure_returns_no_bodies(tools_setup_pd):
    """Discovery shape: name+description+score, no body_markdown,
    no nested sub_skills."""
    response = tools_setup_pd.find_skills("how many funded customers?")

    assert response["protocol"] == "progressive_disclosure_v1"
    for skill in response["skills"]:
        assert "body_markdown" not in skill
        assert "sub_skills" not in skill
        assert skill["kind"] in {"hub", "sub_skill"}
        if skill["kind"] == "sub_skill":
            assert skill["parent_skill_id"] is not None

def test_find_skills_progressive_disclosure_promotes_strong_child(tools_setup_pd):
    """A sub-skill that scores higher than its hub should appear
    above the hub in the flat skills list."""
    # The customer-populations sub-skill embeds 'IsFunded' explicitly
    # while the hub has it only as one paragraph among many - the
    # query 'how many funded customers' should rank the child higher.
    response = tools_setup_pd.find_skills("how many funded customers?")
    skill_ids = [s["id"] for s in response["skills"]]
    pop_idx = skill_ids.index("customer-populations-and-lifecycle")
    hub_idx = skill_ids.index("domain-customer-and-identity")
    assert pop_idx < hub_idx
```

4.3 Validator tests

File: `databricks/data-skills/scripts/tests/test_validate_skills.py` (extend)

```
def test_identity_003_catches_hub_vs_subskill_collision(tmp_path):
    """A sub-skill with the same stem as a hub (or another sub-skill
    in a different hub) must fail IDENTITY-003."""
    # Hub: skills/domain-foo/SKILL.md (stem = 'domain-foo')
    # Sub-skill A: skills/domain-foo/dashboard-queries.md (stem = 'dashboard-queries')
    # Sub-skill B: skills/domain-bar/dashboard-queries.md (stem = 'dashboard-queries')
    # → collision on 'dashboard-queries'
    _make_hub(tmp_path / "skills/domain-foo", with_sub="dashboard-queries")
    _make_hub(tmp_path / "skills/domain-bar", with_sub="dashboard-queries")

    findings = run_validator(tmp_path / "skills")

    collision = [f for f in findings if f.check_id == "IDENTITY-003"]
    assert len(collision) == 1
    assert "dashboard-queries" in collision[0].message
```

Rollout plan

Order

1. **PR #1 (Phase 1)** - sub_k bump, gateway banner, find_skills tool description, IDENTITY-003 union dedupe. Days 0-3. Independent of Phase 2.
2. **PR #2 (Phase 2.1)** - routing_protocol module + Settings + app.py wiring + `_attach_routing_protocol` injection on legacy paths. Behind `SKILLS_ROUTING_PROTOCOL_ENABLED=False` by default. Days 3-7.
3. **PR #3 (Phase 2.2)** - ProgressiveDisclosurePolicy + the new `_build_progressive_disclosure_response` helper + tool description update. Behind `SKILLS_PROGRESSIVE_DISCLOSURE_ENABLED=False` by default. Days 7-14.
4. **Stg flip** - flip both env vars to True in stg, run for 7-14 days, observe audit metrics (no 500s, latency unchanged within +/- 10%, find_skills + get_skill call ratio approaches 1:1).
5. **PR #4 (Phase 3.1)** - default both flags to True in `app.yaml` for prod. Days 21-28.
6. **PR #5 (Phase 3.2, optional)** - retire the bulk-dump legacy path. Days 30+.

Audit columns to monitor

The gateway audit middleware already logs `returned_skill_ids`, `triggering_skill_ids`, `skills_top_score`, `had_matched_sub_skills`, etc. Two new columns to add for the rollout:

- `protocol_shape` - "hub_only" / "sub_pass_v1" / "progressive_disclosure_v1" - tells operators which response path the call took. Extracted from the `protocol` field in the response (or absence -> "hub_only").

- `routing_protocol_attached` - boolean, derived from response containing the field. Useful to verify the routing protocol is reaching clients during the staged rollout.

Rollback

Flip the env var back to False. The off-path returns the byte-for-byte legacy shape - no client code change needed.

Risks and open questions

1. **Backwards compatibility for direct `find_skills` callers.** Anyone consuming the response directly (not just LLMs) will see schema changes when the flags flip. The audit table reads `data.skills[i].id` - that key is preserved. The `body_markdown` key is GONE in progressive-disclosure mode - audit / analytics code that reads it will silently get `KeyError` or `None`. **Action:** `grep body_markdown` across the org's analytics layer before the Phase 3.1 default flip.
 2. **Sub-skills as first-class candidates may surface `customer_exclude_list` style edge cases.** The current sub-pass code applies the union ACL filter (`acl_filter_with_subs`) so a sub-skill doesn't appear unless its hub is visible. The new flat shape preserves this, but operators should sanity-check that sub-skill descriptions are written defensively (no PII, no internal-only language).
 3. **Routing protocol cap at 4000 chars.** The current `databricks/data-rules/.assistant_workspace_instructions.md` is 53 lines (~3KB) - well under the cap. If it grows past 4KB, truncation kicks in and authors lose tail content silently except for a log line. Authors should be warned in the data-rules README.
 4. **`get_skill(id)` activation cost.** Each activation is one extra round-trip vs the current bulk-dump shape. Net token cost is LOWER (one full body vs five hubs + their nested children), but latency adds ~50-200ms per turn for the second tool call. Expected to be invisible to users; worth measuring during staging.
 5. **Genie spaces vs Genie code distinction.** The routing protocol references "PROD - Registration to FTD genie space" - that's a Genie SPACE (curated AI/BI), not Genie CODE. External MCP clients can't invoke a Genie space directly; they'd need to either ask the user or call the Databricks Genie REST API. **Open question for DE: should the routing protocol be filtered for external clients to remove Genie-space-specific routing rules, or should we add an MCP tool that proxies Genie space calls?** Out of scope for this PR but worth flagging.
 6. **`AGENTS.md` / `CLAUDE.md` auto-discovery.** Genie Code also reads these from the directory tree. Out of scope here but a logical Phase 4 if MCP clients want local-context discovery on top of the workspace protocol.
-

What to take to the DE team

The shortest meeting agenda:

1. Agree the diagnosis from the companion PDF ([skills-mcp-routing-failure-analysis-2026-06-03.pdf](#)) - Genie Code wins because of workspace-instructions auto-load + progressive disclosure, not because of better skill content.
2. Approve the phased plan above (4 PRs across ~3-4 weeks).
3. Decide who owns each PR. Phase 1 is small enough for one engineer in a day; Phase 2 is the substantive work.
4. Confirm that [databricks/data-rules/.assistant_workspace_instructions.md](#) is the correct source of truth for the routing protocol that ships to MCP clients (vs forking a separate doc).
5. Decide on the open question in Risk #5 (Genie-space references in the routing protocol).

If the team agrees, the implementation work fits within DEI-3745 (the existing sub-skills routing epic) or warrants a new ticket. Either way, the existing feature-flag rollout pattern is a good model for execution.

Prepared by Cursor (Composer 2.5) on 2026-06-03 as a working specification. All file paths, dataclass names, and method signatures are drawn from the actual repo state at this date and are intended to drop in with minimal modification. Companion document: [skills-mcp-routing-failure-analysis-2026-06-03.pdf](#).